

ETEC PROF CAMARGO ARANHA
ENSINO MÉDIO COM TÉCNICO INTEGRADO A DESENVOLVIMENTO DE
SISTEMAS (M-TEC)

LHORRANY CERQUEIRA SILVA CARDOSO DE OLIVEIRA
MARIA CAROLINA SAYO OGASSAWARA FREITAS
MATHEUS MINAMI PACHECO
RICKELMY FEITOSA MALAFAIA
RODRIGO MATEUS ALVARENGA SILVA

DOCUMENTAÇÃO TÉCNICA DE SOFTWARE - OPEN SOUND

SÃO PAULO
28 DE SETEMBRO DE 2026

SUMÁRIO

1. INTRODUÇÃO.....	3
1.1 Objetivo do documento.....	3
1.2 Escopo do sistema.....	3
1.3 Convenções utilizadas.....	3
2. VISÃO GERAL DO PRODUTO.....	4
2.1 Descrição geral.....	4
2.2 Público-alvo.....	4
2.3 Posicionamento em relação a referências de mercado.....	4
2.4 Estado atual do projeto.....	4
3. ARQUITETURA DO SISTEMA.....	5
3.1 Visão macro.....	5
3.2 Diagrama textual de arquitetura.....	5
3.3 Stack tecnológica.....	5
3.4 Organização de diretórios.....	6
4. MODELAGEM DE DADOS.....	7
4.1 Tabela usuarios.....	7
4.2 Tabela musicas.....	7
4.3 Tabelas de verificação temporária.....	8
4.4 Tabelas de histórico, playlists e perfil.....	8
4.5 Relacionamentos e integridade.....	8
5. SEGURANÇA E AUTENTICAÇÃO.....	9
5.1 Cadastro com verificação em duas etapas (2FA).....	9
5.2 Login e emissão de token (JWT).....	9
5.3 Hash de senhas.....	9
5.4 Recuperação e redefinição de senha.....	10
5.5 Limitação de requisições (rate limiting).....	10
5.6 Armazenamento de arquivos.....	10
5.7 Exclusão de conta.....	10
5.8 Pontos de atenção antes da publicação.....	11
6. REQUISITOS FUNCIONAIS (MVP).....	12
7. API REST - REFERÊNCIA DE ENDPOINTS.....	13
7.1 Autenticação e cadastro.....	13
7.2 Usuário e perfil.....	13
7.3 Músicas.....	14
7.4 Playlists e favoritos.....	14
8. FRONTEND - ESTRUTURA, TAGS E COMPONENTES.....	15
8.1 Barra de navegação (<nav class="fixed-bar">).....	15
8.2 Conteúdo principal (<div class="main-ctt">).....	15
8.3 Modais (<div class="modal">).....	15

8.4 Player fixo (<div id="player-barra">).	16
8.5 Outras páginas.	16
8.6 Convenção de nomenclatura.	16
9. FLUXOS DE USO (CASOS DE USO).	17
9.1 Cadastro de novo usuário.	17
9.2 Login e recuperação de senha.	17
9.3 Tornar-se artista e enviar música.	17
9.4 Reprodução, favoritos e playlists.	17
9.5 Gestão de perfil e conta.	18
10. REQUISITOS NÃO FUNCIONAIS.	19
11. INSTALAÇÃO E EXECUÇÃO (AMBIENTE DE DESENVOLVIMENTO).	20
11.1 Pré-requisitos.	20
11.2 Passo a passo.	20
11.3 Variáveis de ambiente necessárias.	20
12. TESTES.	21
13. CHECKLIST DE PRONTIDÃO PARA LANÇAMENTO.	22
13.1 Já implementado.	22
13.2 Pendente antes da publicação.	22
14. ROADMAP - TRABALHOS FUTUROS.	24
15. REFERÊNCIAS.	25
APÊNDICE A - DIÁRIO DE BORDO (HISTÓRICO DE COMMITS).	26
19/08/2026.	26
09/09/2026.	26
15/09/2026.	26
17/09/2026.	26
21 a 22/09/2026.	26
23/09/2026.	26
24/09/2026.	27
27/09/2026.	27
28/09/2026.	27

1. INTRODUÇÃO

1.1 Objetivo do documento

Este documento descreve, de forma técnica e organizada, o funcionamento do software OpenSound: sua arquitetura, modelagem de dados, regras de negócio, endpoints de API, estrutura de frontend e os requisitos considerados necessários para que o sistema evolua de um protótipo funcional (MVP) para uma versão apta a ser publicada. O documento serve tanto como referência para a equipe de desenvolvimento quanto como material de apoio para avaliação escolar.

1.2 Escopo do sistema

O OpenSound é uma aplicação web que permite que usuários criem conta, façam login autenticado e, ao se cadastrarem como artistas, enviem (upload) arquivos de música, disponibilizando-os para reprodução por outros usuários cadastrados. Além do catálogo, o sistema oferece busca, ranking de músicas e artistas mais ouvidos, playlists (incluindo a playlist automática “Favoritos”), perfil de usuário e recuperação de senha. O catálogo de músicas é público para navegação, mas a reprodução exige autenticação. O sistema é inspirado em plataformas de streaming e upload como Spotify e SoundCloud, sem vínculo comercial ou institucional com essas empresas.

1.3 Convenções utilizadas

Nomes de rotas, tabelas, variáveis e arquivos são citados no formato de código (ex.: `server.js`). Requisitos funcionais são identificados pelo prefixo RF; requisitos não funcionais, pelo prefixo RNF.

2. VISÃO GERAL DO PRODUTO

2.1 Descrição geral

O OpenSound resolve o problema de compartilhamento simples e rápido de faixas musicais: qualquer usuário que tenha registrado um nome de artista pode publicar uma música (com título, arquivo de áudio e capa opcional) sem passar por um processo de aprovação prévia, e o catálogo é atualizado em tempo real para todos os visitantes. A proposta central é reduzir o atrito para novos artistas e ouvintes, priorizando facilidade de uso sobre curadoria.

2.2 Público-alvo

Artistas independentes e criadores de conteúdo que desejam publicar faixas sem burocracia; disponibilizar a oportunidade de publicar faixas específicas que existem em apenas um streaming, como por exemplo versões de músicas ao vivo ou covers; ouvintes que buscam descobrir músicas recentes; e, no contexto atual do projeto, avaliadores acadêmicos e colaboradores da equipe de desenvolvimento.

2.3 Posicionamento em relação a referências de mercado

Diferente do Spotify (modelo majoritariamente fechado, com catálogo curado por selos e distribuidoras), o OpenSound segue o modelo aberto de upload direto do SoundCloud, no qual o próprio usuário é responsável pelo conteúdo publicado. Essa escolha de produto tem uma implicação jurídica direta, tratada em detalhe no Termo de Licenciamento e Uso: a plataforma não realiza checagem prévia de direitos autorais sobre as faixas enviadas.

2.4 Estado atual do projeto

O sistema encontra-se em estágio de MVP (Produto Mínimo Viável) funcional: cadastro com verificação em duas etapas, login, recuperação e redefinição de senha, cadastro de artista, upload de músicas, listagem pública, busca, rankings de músicas e artistas, playlists, favoritos, perfil de usuário e exclusão de conta já estão implementados. Funcionalidades como histórico de escuta, filtro por gênero, hierarquia de playlists e monetização estão planejadas para etapas futuras (ver seção 14).

3. ARQUITETURA DO SISTEMA

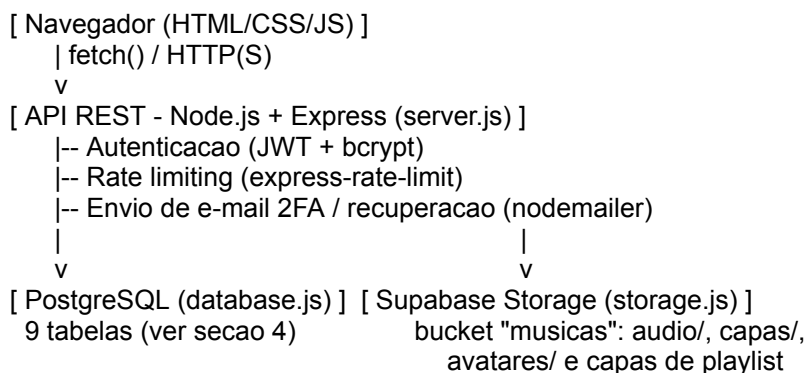
3.1 Visão macro

O OpenSound segue uma arquitetura cliente-servidor de três camadas:

- Camada de apresentação (frontend): HTML, CSS e JavaScript vanilla (sem framework), executados no navegador do usuário, em quatro páginas.
- Camada de aplicação (backend): API REST em Node.js com Express, responsável pelas regras de negócio, autenticação e validação.
- Camada de dados: banco relacional PostgreSQL para dados estruturados e Supabase Storage (armazenamento de objetos) para os arquivos binários de áudio, capas e avatares.

Fluxo simplificado: o navegador consome a API REST via `fetch()`; o backend valida a requisição, aplica autenticação/autorização quando necessário, consulta ou grava no PostgreSQL e, quando há arquivo, envia o binário ao Supabase Storage, devolvendo ao frontend apenas metadados e URLs públicas.

3.2 Diagrama textual de arquitetura



3.3 Stack tecnológica

Camada	Tecnologia	Função no projeto
Frontend	HTML5 / CSS3 / JavaScript (ES6+)	Interface, modais, player, perfil e chamadas à API
Backend	Node.js + Express 5	Servidor HTTP e rotas da API REST
Banco de dados	PostgreSQL (pacote pg)	Persistência de usuários, músicas, playlists, perfil e verificações

Camada	Tecnologia	Função no projeto
Armazenamento	Supabase Storage (@supabase/supabase-js)	Arquivos de áudio, capas e avatares
Autenticação	JWT (jsonwebtoken) + bcrypt	Token de sessão e hash de senha/códigos
E-mail	Nodemailer (SMTP Gmail)	Códigos de verificação (cadastro e recuperação de senha)
Upload	Multer (memória)	Recebe multipart/form-data e repassa ao Storage
Tráfego	express-rate-limit, cors	Limitação de requisições por IP e controle de origem
Configuração	dotenv	Variáveis de ambiente (.env)

3.4 Organização de diretórios

```

open-sound/
+-- backend/
|  +-- server.js    -> rotas da API e regras de negócio
|  +-- database.js  -> conexao e criacao/migracao das tabelas
|  +-- storage.js   -> integracao com o Supabase Storage
|  +-- .env.example -> modelo das variaveis de ambiente
|  +-- package.json -> dependencias do backend
+-- frontend/
|  +-- html/ home, perfil, minhas-musicas, config
|  +-- css/  root, theme, home, perfil
|  +-- js/   home, perfil, minhas-musicas, config
+-- project docs/ -> politica de privacidade, termos de licenciamento e uso,
|                  MVP, stack e dependencias
+-- README.md

```

4. MODELAGEM DE DADOS

O banco relacional é criado automaticamente na inicialização do servidor (função `criarTabelas` em `database.js`), por meio de instruções `CREATE TABLE IF NOT EXISTS`. Como essa instrução não altera tabelas já existentes, colunas adicionadas depois são aplicadas por migrações leves (`ALTER TABLE ... ADD COLUMN IF NOT EXISTS`), executadas a cada inicialização sem apagar dados.

4.1 Tabela usuarios

Campo	Tipo	Regras
id	SERIAL	Chave primária
email	VARCHAR(255)	Único, obrigatório
senha	VARCHAR(255)	Hash bcrypt - nunca texto puro
nome_usuario	VARCHAR(255)	Nome de exibição, informado no cadastro
verificado	BOOLEAN	Padrão FALSE; TRUE após o 2FA
eh_artista	BOOLEAN	Padrão FALSE; TRUE após registrar nome de artista
nome_artista	VARCHAR(255)	Nome usado como autor das músicas
bio	TEXT	Biografia do perfil (máx. 300 caracteres)
url_avatar	TEXT	URL pública do avatar
musica_favorita_id	INTEGER	FK -> musicas(id); ON DELETE SET NULL
senha_redefinida_em	TIMESTAMP	Data da última redefinição de senha
criado_em	TIMESTAMP	Data/hora de criação

4.2 Tabela musicas

Campo	Tipo	Regras
id	SERIAL	Chave primária
titulo	VARCHAR(255)	Obrigatório
artista	VARCHAR(255)	Obrigatório; copiado de usuarios.nome_artista
url_audio	TEXT	URL pública do áudio; obrigatório
url_capa	TEXT	URL pública da capa; pode ser nula
usuario_id	INTEGER	FK -> usuarios(id)
reproducoes	INTEGER	Contador de plays; padrão 0
criado_em	TIMESTAMP	Ordena o catálogo (mais recentes primeiro)

4.3 Tabelas de verificação temporária

Três tabelas funcionam como “área de espera” de códigos enviados por e-mail, todas com código armazenado apenas em hash, contador de tentativas (máx. 5), validade de 10 minutos e controle de reenvio (ultimo_envio_em, cooldown de 60 s):

Tabela	Finalidade	Campos específicos
verificacoes_2fa	Cadastro pendente (id UUID)	email, senha_hash, nome_usuario, codigo_hash
recuperacoes_senha	“Esqueci minha senha” (sem login)	email, codigo_hash, verificado (TRUE após validar o código)
redefinicoes_senha	Troca de senha por usuário logado	usuario_id (FK), nova_senha_hash, codigo_hash

4.4 Tabelas de histórico, playlists e perfil

Tabela	Finalidade	Campos principais
historico_redefinicoes_senha	Registra cada redefinição concluída; base do limite de 3 em 24 h pelo modal	usuario_id (CASCADE), origem (“modal” ou “config”), criado_em
playlists	Playlists do usuário; “Favoritos” é criada sob demanda	nome, url_capa, usuario_id, eh_favoritos
playlist_musicas	Junção playlist-música (sem duplicatas)	playlist_id, musica_id (CASCADE), UNIQUE(playlist_id, musica_id)
perfil_curtidas	Até 4 músicas em destaque no perfil	usuario_id, musica_id (CASCADE), posicao, UNIQUE(usuario_id, musica_id)

4.5 Relacionamentos e integridade

musicas.usuario_id referencia usuarios.id, associando cada faixa ao seu autor. As tabelas de verificação são independentes: o registro definitivo em usuarios só é criado após a confirmação do código, evitando cadastros incompletos. Tabelas dependentes (playlist_musicas, perfil_curtidas, historico_redefinicoes_senha) usam ON DELETE CASCADE, e a exclusão de conta é feita em transação (seção 5.7).

5. SEGURANÇA E AUTENTICAÇÃO

5.1 Cadastro com verificação em duas etapas (2FA)

O cadastro não cria o usuário imediatamente. O fluxo é:

- 1. O usuário envia e-mail, nome de usuário e senha para POST /api/registro.
- 2. O backend valida formato de e-mail e força da senha, verifica duplicidade, gera um código numérico de 6 dígitos e grava apenas o hash desse código na tabela verificacoes_2fa, com expiração de 10 minutos.
- 3. O código é enviado por e-mail via Nodemailer.
- 4. O usuário confirma o código em POST /api/validar-2fa; ao atingir 5 tentativas incorretas, a verificação é descartada e um novo cadastro deve ser iniciado.
- 5. Somente após a confirmação o registro definitivo é criado em usuarios.

A senha deve ter 8 ou mais caracteres, com maiúscula, minúscula, número e ao menos um caractere especial do conjunto @ () ! % * ? & #; outros caracteres são recusados pela expressão regular atual.

5.2 Login e emissão de token (JWT)

O login (POST /api/login) confere e-mail e senha (comparação de hash com bcrypt) e, em caso de sucesso, emite um JSON Web Token assinado com JWT_SECRET_SESSAO, carregando apenas dados de identificação do usuário, nunca o hash da senha. A validade é de 7 dias por padrão ou 30 dias com a opção lembrarDeMim. Rotas protegidas exigem o cabeçalho Authorization: Bearer <token>, validado pelo middleware verificarAutenticacao.

5.3 Hash de senhas

Senhas nunca são armazenadas em texto puro: o backend usa bcrypt tanto para a senha de conta quanto para os códigos de verificação (2FA, recuperação e redefinição), o que impede a recuperação do valor original mesmo em caso de vazamento do banco de dados.

5.4 Recuperação e redefinição de senha

- Esqueci minha senha (sem login): esqueci-senha gera o código, validar-esqueci marca a recuperação como verificada e confirmar-esqueci grava a nova senha. Limite de 3 redefinições concluídas por conta em 24 horas, recheado na confirmação para impedir burla por várias sessões.
- Redefinição logada (página de configurações): solicitar e confirmar exigem token e código enviado por e-mail.

5.5 Limitação de requisições (rate limiting)

Rota	Janela	Limite por IP
POST /api/registro	15 minutos	5
POST /api/validar-2fa	15 minutos	10
POST /api/reenviar-2fa	1 hora (+ cooldown de 60 s)	5
POST /api/login	15 minutos	10
POST /api/auth/esqueci-senha, validar-esqueci e confirmar-esqueci	15 minutos	5
POST /api/auth/reenviar-esqueci	1 hora	5
POST /api/usuarios/redefinir-senha/*	15 minutos	10
POST /api/musicas (upload)	1 hora	20

Essa camada mitiga ataques de força bruta contra login e códigos, e reduz abuso do envio de e-mails e de uploads a partir de um mesmo IP.

5.6 Armazenamento de arquivos

Arquivos são recebidos em memória pelo Multer (nunca gravados em disco local, o que mantém o backend compatível com ambientes de hospedagem efêmeros) e repassados ao bucket “musicas” do Supabase Storage, nas pastas audio/, capas/ e avatares/. Limites: 25 MB por arquivo de música (áudio MP3/WAV/OGG; capa JPEG/PNG/WEBP), 5 MB para capa de playlist e 2 MB para avatar (JPEG/PNG/WEBP). Nomes de avatar são gerados pelo servidor; nomes de áudio e capa recebem prefixo único.

5.7 Exclusão de conta

DELETE /api/usuarios/eu remove os arquivos do usuário no Storage (falhas nessa etapa são registradas, mas não impedem a exclusão) e, em transação, apaga

músicas, playlists, redefinições pendentes e a conta, com ROLLBACK em caso de erro. Atende ao direito de eliminação previsto na LGPD.

5.8 Pontos de atenção antes da publicação

- A verificação de certificado TLS do serviço de e-mail está desativada (`tls.rejectUnauthorized: false`); deve ser removida antes da publicação.
- A conexão com o PostgreSQL também usa `ssl: { rejectUnauthorized: false }` em `database.js`; recomenda-se validar o certificado em produção.
- O bucket de armazenamento é público: qualquer pessoa com a URL acessa o arquivo, sem token. Considerar bucket privado com URLs assinadas.
- O CORS está liberado para qualquer origem (`app.use(cors())`); restringir ao domínio do frontend em produção.
- A resposta de login diferencia “e-mail não cadastrado” de “senha incorreta”, o que permite enumeração de contas; trade-off consciente a reavaliar.
- As URLs da API estão fixas como `http://localhost:3000` em todos os arquivos JavaScript do frontend; migrar para configuração única.
- O upload de áudio/capa confia no tipo MIME informado pelo cliente; considerar validação do conteúdo real do arquivo.

6. REQUISITOS FUNCIONAIS (MVP)

Código	Requisito	Status
RF01	Cadastro de usuário com e-mail, nome de usuário e senha	Implementado
RF02	Verificação em duas etapas (2FA) por e-mail no cadastro	Implementado
RF03	Reenvio do código 2FA, respeitando cooldown	Implementado
RF04	Login com e-mail e senha, com opção “lembrar de mim”	Implementado
RF05	Logout, encerrando a sessão local	Implementado
RF06	Upload de música (título, áudio, capa opcional) apenas para usuários autenticados que sejam artistas	Implementado
RF07	Listagem pública do catálogo, mais recentes primeiro	Implementado
RF08	Exigir login para reproduzir (navegar é livre; tocar não)	Implementado
RF09	Alternância de tema claro/escuro	Implementado
RF10	Busca por música e artista	Implementado
RF11	Recomendações: músicas e artistas mais ouvidos	Implementado
RF12	Criação e organização de playlists (criar, listar, excluir, adicionar/remover músicas, capa)	Implementado (sem hierarquia)
RF13	Perfil de usuário com foto, biografia, música favorita e músicas curtidas	Implementado (sem histórico)
RF14	Recuperação de senha (“esqueci minha senha”) por código enviado ao e-mail	Implementado
RF15	Redefinição de senha por usuário logado, nas configurações	Implementado
RF16	Cadastro de nome de artista para habilitar uploads	Implementado
RF17	Contagem de reproduções por música	Implementado
RF18	Favoritar músicas (playlist automática “Favoritos”)	Implementado
RF19	Listar e excluir as próprias músicas (“Minhas Músicas”)	Implementado
RF20	Exclusão da conta e de todos os dados associados	Implementado
RF21	Filtro por gênero	Backlog
RF22	Histórico de escuta	Backlog

7. API REST - REFERÊNCIA DE ENDPOINTS

Todas as respostas seguem o formato JSON com o campo status (“sucesso” ou “erro”) e mensagem descritiva. Rotas autenticadas exigem Authorization: Bearer <tokenSessao>.

7.1 Autenticação e cadastro

Método e rota	Auth	Descrição
POST /api/registro	Não	Corpo: email, password, nomeUsuario. Retorna idVerificacao. Erros: 400, 429
POST /api/validar-2fa	Não	Corpo: codigo, idVerificacao. Cria e ativa a conta. Erros: 400, 429
POST /api/reenviar-2fa	Não	Corpo: idVerificacao. Novo código (cooldown 60 s). Erros: 400, 429
POST /api/login	Não	Corpo: email, password, lembrarDeMim. Retorna tokenSessao. Erros: 401, 403, 404, 429
POST /api/auth/esqueci-senha	Não	Inicia a recuperação e envia o código
POST /api/auth/reenviar-esqueci	Não	Reenvia o código de recuperação
POST /api/auth/validar-esqueci	Não	Valida o código de recuperação
POST /api/auth/confirmar-esqueci	Não	Grava a nova senha (limite de 3 em 24 h)

7.2 Usuário e perfil

Método e rota	Auth	Descrição
GET /api/usuarios/eu	Sim	Dados da conta logada
POST /api/usuarios/artista	Sim	Corpo: nomeArtista. Habilita upload
GET /api/usuarios/eu/perfil	Sim	Dados do perfil, favorita e curtidas
PUT /api/usuarios/eu/bio	Sim	Biografia (até 300 caracteres)
POST /api/usuarios/eu/avatar	Sim	multipart, campo avatar (até 2 MB)
PUT /api/usuarios/eu/favorita	Sim	Define a música favorita
PUT /api/usuarios/eu/curtidas	Sim	Define até 4 músicas em destaque
POST /api/usuarios/redefinir-senha/solicitar	Sim	Envia código para trocar a senha
POST /api/usuarios/redefinir-senha/confirmar	Sim	Confirma o código e grava a nova senha
DELETE /api/usuarios/eu	Sim	Exclui a conta e seus dados

7.3 Músicas

Método e rota	Auth	Descrição
POST /api/musicas	Sim (artista)	multipart: titulo, audio, capa (opcional); o artista vem do perfil. Erros: 400, 401, 403 (não é artista), 429, 500
GET /api/musicas	Não	50 músicas mais recentes
GET /api/musicas/mais-tocadas	Não	10 músicas com mais reproduções
GET /api/musicas/buscar?q=	Não	Busca por título/artista (até 50)
GET /api/artistas/mais-ouvidos	Não	10 artistas por reproduções
GET /api/musicas/minhas	Sim	Músicas do usuário logado
POST /api/musicas/:id/reproduzir	Sim	Incrementa o contador de reproduções
DELETE /api/musicas/:id	Sim	Exclui música própria (403 se de outro usuário)

7.4 Playlists e favoritos

Método e rota	Auth	Descrição
GET /api/playlists	Sim	Lista playlists com total de músicas
POST /api/playlists	Sim	Cria playlist (nome; capa opcional até 5 MB)
GET /api/playlists/:id	Sim	Detalhe e músicas da playlist
DELETE /api/playlists/:id	Sim	Exclui playlist
POST /api/playlists/:id/musicas	Sim	Corpo: musicalId
DELETE /api/playlists/:id/musicas/:musicalId	Sim	Remove música da playlist
POST e DELETE /api/musicas/:id/favoritar	Sim	Adiciona/remove de Favoritos
GET /api/musicas/favoritos/ids	Sim	IDs das músicas favoritadas

8. FRONTEND - ESTRUTURA, TAGS E COMPONENTES

O frontend não usa framework: é HTML, CSS e JavaScript organizados por responsabilidade, em quatro páginas: home.html, perfil.html, minhas-musicas.html e config.html. Os estilos ficam em root.css (variáveis), theme.css (tema claro/escuro), home.css e perfil.css.

8.1 Barra de navegação (<nav class="fixed-bar">)

- nav-left: menu hambúrguer (“Minhas Músicas” e “Configurações”) e botões “Minha Biblioteca”, “Início” e “Upload”.
- nav-center: formulário de busca com `<input type="search" name="q">`.
- nav-right: “registre-se”, “Entrar”, “Sair” (oculto quando deslogado) e alternância de tema.

8.2 Conteúdo principal (<div class="main-ctt">)

- #lista-musicas: carrossel com setas, preenchido por GET /api/musicas/mais-tocadas; se não houver reproduções, usa o catálogo geral. Cada música vira um card.
- .double-div: #lista-artistas (artistas mais ouvidos, dinâmico) e .genres (“Descubra novos gostos”, ainda sem conteúdo).

8.3 Modais (<div class="modal">)

- #modal-registro: duas etapas - formulário com regras de senha em tempo real e grade de 6 input-otp com foco automático e suporte a colar o código.
- #modal-login: e-mail, senha e “lembrar de mim por 30 dias”.
- #modal-esqueci-senha: fluxo de recuperação em etapas (e-mail, código, nova senha).
- #modal-artista: cadastro do nome de artista, exigido antes do primeiro upload.
- #modal-upload: envio de música com pré-visualização ao vivo de título, artista e capa.

- #modal-add-playlist: escolha da playlist (#lista-playlists-modal) para adicionar a faixa.

8.4 Player fixo (<div id="player-barra">)

Barra fixa na base da tela, oculta até a primeira reprodução. Contém capa, título, artista, play/pause, tempo decorrido, barra de progresso (#player-seek), tempo total, botão de favoritar (#player-favoritar) e botão de adicionar à playlist (#player-add-playlist). É controlada pelo home.js a partir de um elemento <audio> HTML5.

8.5 Outras páginas

- perfil.html: avatar, biografia, música favorita e até 4 músicas curtidas.
- minhas-musicas.html: lista e exclusão das músicas do próprio usuário.
- config.html: redefinição de senha (com código por e-mail) e exclusão de conta.

8.6 Convenção de nomenclatura

IDs e classes seguem nomenclatura em português, alinhada ao restante do código (ex.: btn-entrar, campo-senha, mensagem-lista). Manter esse padrão em novas telas evita inconsistência entre arquivos.

9. FLUXOS DE USO (CASOS DE USO)

9.1 Cadastro de novo usuário

- 1. Usuário clica em “registre-se” e preenche e-mail, nome de usuário e senha (regras validadas em tempo real).
- 2. O sistema envia código de 6 dígitos por e-mail e abre a etapa de verificação.
- 3. Usuário digita ou cola o código; o sistema confirma e ativa a conta.
- 4. Alternativa: se o e-mail já existir, o cadastro é interrompido e abre-se o modal de login pré-preenchido.

9.2 Login e recuperação de senha

- 1. Usuário informa e-mail e senha, opcionalmente marca “lembrar de mim”; o token de sessão é guardado no navegador.
- 2. Se esqueceu a senha, abre o modal de recuperação, valida o código enviado por e-mail e define nova senha.

9.3 Tornar-se artista e enviar música

- 1. Usuário autenticado clica em “Upload”; se não estiver logado, é solicitado o login.
- 2. Se ainda não for artista, informa o nome de artista no #modal-artista.
- 3. Preenche título, seleciona áudio (obrigatório) e capa (opcional), acompanhando a pré-visualização.
- 4. O backend valida, grava os arquivos no Storage e cria o registro; o catálogo é recarregado.

9.4 Reprodução, favoritos e playlists

- 1. Qualquer visitante navega pelo catálogo; ao clicar em “tocar”, sem sessão ativa, o login é solicitado.
- 2. Ao tocar, a reprodução é contabilizada em POST /api/musicas/:id/reproduzir.
- 3. No player, o usuário pode favoritar a faixa ou adicioná-la a uma playlist.

9.5 Gestão de perfil e conta

- 1. Em “perfil”, o usuário edita avatar, biografia, música favorita e curtidas.
- 2. Em “Minhas Músicas”, exclui faixas próprias; em “Configurações”, redefine a senha ou exclui a conta.

10. REQUISITOS NÃO FUNCIONAIS

Código	Categoria	Descrição
RNF01	Segurança	Senhas e códigos nunca armazenados em texto puro (hash bcrypt)
RNF02	Segurança	Toda rota sensível exige token JWT válido, validado no backend
RNF03	Disponibilidade	Limite de requisições por IP para reduzir risco de abuso
RNF04	Usabilidade	Erros em linguagem clara, em português, em todas as chamadas da API
RNF05	Portabilidade	Backend sem dependência de armazenamento em disco local
RNF06	Manutenibilidade	Separação entre rotas (server.js), dados (database.js) e arquivos (storage.js)
RNF07	Conformidade legal	Tratamento de dados pessoais alinhado à LGPD (Lei nº 13.709/2018)
RNF08	Integridade	Migrações de esquema invariante à repetição e exclusão de conta em transação

11. INSTALAÇÃO E EXECUÇÃO (AMBIENTE DE DESENVOLVIMENTO)

11.1 Pré-requisitos

- Node.js instalado.
- Instância de PostgreSQL acessível.
- Projeto no Supabase, com Storage habilitado.
- Conta Gmail para envio dos códigos via Nodemailer.

11.2 Passo a passo

- 1. Copiar .env.example para .env e preencher as credenciais.
- 2. Abrir o terminal na pasta backend.
- 3. Executar npm install.
- 4. Executar npm start (ou node server.js).
- 5. Abrir frontend/html/home.html no navegador (ou servir a pasta por servidor estático).
- 6. Encerrar o servidor com Ctrl + C.

11.3 Variáveis de ambiente necessárias

Variável	Finalidade
DB_HOST, DB_PORT, DB_USER, DB_PASSWORD, DB_NAME	Conexão com o PostgreSQL
SUPABASE_URL, SUPABASE_SERVICE_ROLE_KEY	Acesso administrativo ao Storage (uso exclusivo do backend)
SUPABASE_BUCKET_MUSICAS	Nome do bucket (padrão: "musicas")
GMAIL_USER, GMAIL_PASS	Conta usada pelo Nodemailer
JWT_SECRET_SESSAO	Segredo dos tokens - obrigatório, sem valor padrão
PORT	Porta HTTP (padrão: 3000)

A ausência de JWT_SECRET_SESSAO, SUPABASE_URL ou SUPABASE_SERVICE_ROLE_KEY interrompe a inicialização propositalmente, evitando subir a aplicação com segredo previsível ou sem acesso ao armazenamento.

12. TESTES

O projeto ainda não possui testes automatizados (o script de teste do package.json é um placeholder). Para uma versão pronta para lançamento, recomenda-se:

- Testes unitários das funções de validação (validarEmail, validarSenhaForte).
- Testes de integração de cada grupo de endpoints (cadastro/2FA, login, recuperação de senha, perfil, músicas, playlists e favoritos), incluindo cenários de erro (rate limit, token inválido, usuário não artista, arquivo não suportado).
- Testes manuais do fluxo completo (cadastro → login → tornar-se artista → upload → reprodução → playlist) em pelo menos dois navegadores.
- Testes de carga simples nos endpoints públicos de listagem, busca e ranking.

13. CHECKLIST DE PRONTIDÃO PARA LANÇAMENTO

13.1 Já implementado

- Cadastro, 2FA, login e sessão via JWT; recuperação e redefinição de senha.
- Upload autenticado por artistas, com validação de tipo e tamanho.
- Catálogo, busca e rankings de músicas e artistas.
- Playlists, favoritos e perfil de usuário.
- Exclusão de conta com remoção de arquivos.
- Limitação de requisições por IP nas rotas sensíveis.
- Política de Privacidade v1.1 (LGPD) e Termo de Licenciamento e Uso v1.1 revisados conforme o código e mantidos em “project docs”.

13.2 Pendente antes da publicação

- Remover os contornos de TLS (e-mail e banco) e validar certificados.
- Definir a URL da API por configuração, em vez de valor fixo no frontend.
- Restringir o CORS ao domínio do frontend.
- Avaliar bucket privado com URLs assinadas.
- Vincular a aceitação do Termo de Licenciamento e Uso e da Política de Privacidade ao fluxo de cadastro (o checkbox atual em `home.html` ainda é um placeholder, sem link nem registro do aceite).
- Criar e monitorar as caixas `direitos@`, `suporte@` e `privacidade@opensound.com` e implementar o fluxo de denúncia e remoção previsto na seção 5.1 do Termo.
- Confirmar com a equipe a rotina de backups, o provedor de hospedagem do banco e o uso de HTTPS, itens marcados como [confirmar] na Política de Privacidade.
- Decidir se haverá verificação de idade mínima (13 anos), hoje apenas declaratória.

- Garantir a unicidade do nome de usuário: o histórico de commits (23/09) a cita, mas database.js não define UNIQUE em nome_usuario e POST /api/registro só checa duplicidade de e-mail.
- Implementar suíte de testes automatizados mínima.
- Definir processo de moderação/denúncia de conteúdo, dado que não há checagem prévia de direitos autorais.
- Configurar monitoramento, logs centralizados e backup do banco.

14. ROADMAP - TRABALHOS FUTUROS

Itens fora do escopo do MVP atual, mapeados para próximas etapas:

- Filtro por gênero (área “Descubra novos gostos”).
- Perfil ampliado: banner e tema padrão.
- Playlists com hierarquia e fixação de itens.
- Histórico de escuta e filtros inteligentes de recomendação.
- Comentários em músicas e status de usuário (rede social leve).
- Selo de “artista verificado”.
- Monetização: assinatura premium sem anúncios e repasse percentual aos artistas mais ouvidos.
- Relatório periódico de estatísticas de escuta (retrospectiva anual, minutos ouvidos, artistas mais tocados).

15. REFERÊNCIAS

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. NBR 14724: informação e documentação - trabalhos acadêmicos - apresentação. Rio de Janeiro: ABNT, 2011.

BRASIL. Lei nº 12.965, de 23 de abril de 2014. Marco Civil da Internet. Diário Oficial da União, Brasília, DF, 2014.

BRASIL. Lei nº 13.709, de 14 de agosto de 2018. Lei Geral de Proteção de Dados Pessoais (LGPD). Diário Oficial da União, Brasília, DF, 2018.

BRASIL. Lei nº 9.610, de 19 de fevereiro de 1998. Lei de Direitos Autorais. Diário Oficial da União, Brasília, DF, 1998.

EXPRESS.JS. Express - Node.js web application framework. Disponível em: <https://expressjs.com>. Acesso em: 21 set. 2026.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. PostgreSQL Documentation. Disponível em: <https://www.postgresql.org/docs/>. Acesso em: 21 set. 2026.

JSON WEB TOKEN. Introduction to JSON Web Tokens. Disponível em: <https://jwt.io/introduction>. Acesso em: 21 set. 2026.

SUPABASE. Supabase Storage Documentation. Disponível em: <https://supabase.com/docs/guides/storage>. Acesso em: 21 set. 2026.

APÊNDICE A - DIÁRIO DE BORDO (HISTÓRICO DE COMMITS)

Resumo cronológico do desenvolvimento, elaborado a partir do log do repositório (37 commits, de 19/08 a 28/09/2026). Mensagens repetidas foram agrupadas.

19/08/2026

- Commits iniciais, .env e banco no Supabase, atualização de dependências.txt.
- Primeiro formulário de registro; filtros de senha e e-mail (19 a 21/08).

09/09/2026

- Primeiras versões da Política de Privacidade e dos Termos de Uso.

15/09/2026

- bcrypt para hash de senhas e correção do schema do banco.
- Login real com sessão persistente e campo de código 2FA no cadastro.
- Resolução de conflitos com alterações locais.

17/09/2026

- Envio de músicas, player com minutagem selecionável e capa de álbum; integração com o Supabase Storage.
- Refatoração: o JWT temporário do 2FA foi substituído por verificação armazenada no banco (verificacoes_2fa).

21 a 22/09/2026

- Nome de artista e upload restrito a artistas.
- Criação de root.css e theme.css e melhoria visual da home.

23/09/2026

- Página de configurações, nome de usuário, redefinição de senha e exclusão de conta.
- Aba “Minhas Músicas” (listar e excluir); carrossel e modo claro; merge da branch main.

24/09/2026

- Ajustes de estilo de botões e cores de tema.

27/09/2026

- Busca de músicas pela barra de pesquisa.
- Página de perfil 1.0; carrossel de músicas finalizado.
- Contagem de reproduções e ranking de mais tocadas; limpeza de estilos.

28/09/2026

- Estilização do cadastro e do login; lógica de “esqueci minha senha” no modal.
- Endpoints de playlists e botões no player; correções em redefinição de senha e exclusão de conta.
- Redefinição usa o mesmo timestamp para bloqueio; limite de 3 redefinições por dia pelo modal.
- Favoritos por modal e botão; atalho para criar playlists na biblioteca.
- Remoção do termos-de-uso.md, que estava incorreto.

----- Preciso colocar aqila outra pagina do orientador rs e formatar mais algumas coisas